

CSE498 Project 3: Language-Guided TurtleBot3 Navigation with LocateAnything

Ling Shen

October 4, 2026

Project repository: <https://github.com/lllingshen/project3a>
Demo video directory: [demo_video/](#)

1 System overview

This project adds LocateAnything to the course’s TurtleBot3 navigation system so that a command can select an object by its appearance. Gazebo simulates the room, robot, camera, and laser scanner. ROS 2 nodes use the sensor data to build a map, locate objects, and navigate with Nav2.

A user publishes a string to `/user_command`, for example *Move to the red chair*. The coordinator pauses exploration, cancels conflicting navigation, waits for Nav2 to become idle, and forwards the command. A launch parameter selects exactly one target-selection mode. Both modes reuse the same target message, approach-pose adapter, coordinator, and Nav2 backend.

User command → coordinator: pause and select mode	
Baseline	LocateAnything
Category/index parser + semantic landmarks	Full description + stationary RGB snapshot
YOLO detections feed object memory	Model box + matched LiDAR + saved TF
Selected instance in <code>map</code> → approach pose → Nav2	
SLAM occupancy map + robot pose support planning and motion	

In baseline mode, the parser recognizes configured categories, aliases such as “trash can,” and an optional index such as `person_2`. Without an index, it selects the nearest matching landmark relative to the robot’s current map position. An index selects that exact observed memory ID. IDs reflect observation order, not personal identity. The parser does not model color: both red-chair and blue-chair descriptions reduce to category `chair`. The live comparison is discussed in Section 4.

The detector loads the course’s `tb3det_yolo26n.pt` once; `infer()` returns label, confidence, and pixel bounding box in the wrapper’s existing format. Its classes are `person`, `trash_can`, and `chair`, with a detection threshold of 0.40. The detector publishes boxes and a debug image through the existing ROS wrapper. A direct test of the original stub on a blank 480×640 image returned no detections and `is_loaded=False`.

The project is based on [Yiyuchen Hu’s course repository](#) (a048d1e50e18). I reused detector methods, box parsing, and geometry helpers from [my earlier research repository](#) (694fff69d878). The model runs through [NVIDIA Eagle](#) (8442db3b79f7) with local [LocateAnything-3B](#) weights. The course history and licenses are retained. The chair mesh is by Wan Yi Seow/Open Robotics under CC BY 4.0; the red and blue material changes are listed in NOTICE. Full source commits and run records are in `report/results.json`.

2 Visual grounding and navigation

SLAM Toolbox uses laser scans and robot motion to estimate pose and construct an occupancy grid: cells indicate free, occupied, or unknown space. Semantic memory adds labeled object landmarks such as `person_2`. It associates repeated detections and retains map positions. The occupancy grid supports path planning, while semantic memory connects object labels to locations in that grid.

In LocateAnything mode, the baseline query node is not launched. A persistent worker in a separate Python environment loads the existing local LocateAnything-3B model. The complete referring expression is preserved: only the leading movement phrase is removed. For the red command the actual prompt was:

Locate a single instance that matches the following description: the red chair.

The selected box is localized and sent directly to navigation. YOLO can continue updating semantic memory during this process without changing the selected target. Each selection has its own `locateanything_N` ID.

Before inference, the robot stops and the node captures a camera image, `CameraInfo`, a nearby laser scan, odometry, and timestamped transforms. The sensor matching tolerance is 0.15 s; stale images and motion during inference are rejected. The image keeps its original timestamp, and the transforms are saved before inference so that localization uses the pose at capture time.

Model coordinates use a 0–1000 scale. They are converted to the original $W \times H$ camera pixels by $u = xW/1000$, $v = yH/1000$. For box center (u_c, v_c) and camera intrinsics, the optical ray is

$$\mathbf{r}_c = ((u_c - c_x)/f_x, (v_c - c_y)/f_y, 1).$$

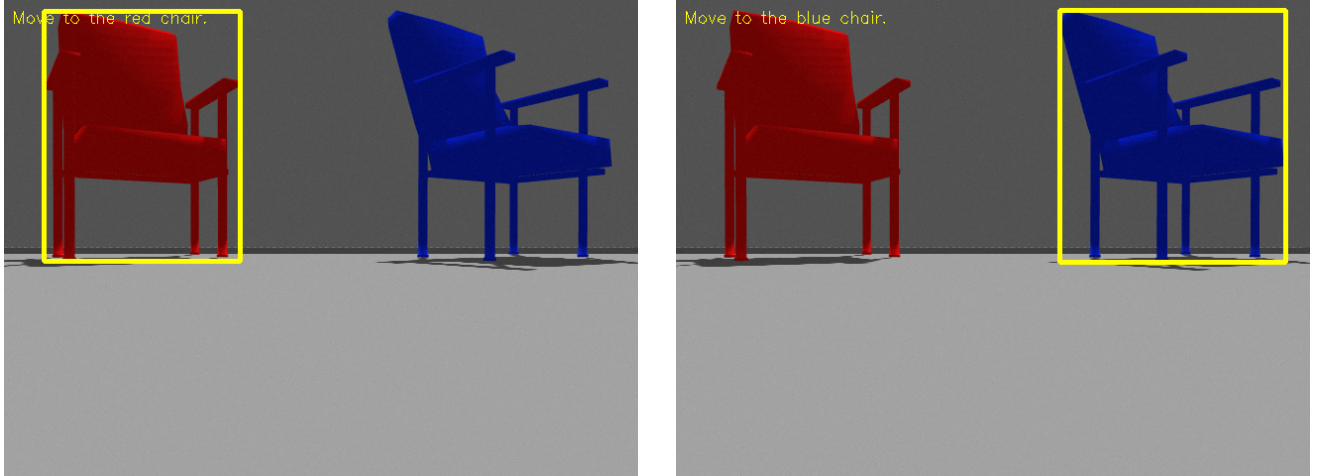
The camera-to-scan rotation gives a planar bearing θ . Nearby valid laser returns inside the box’s angular width provide a median range d . The scan-frame point $(d \cos \theta, d \sin \theta, 0)$ is transformed into `map` using the saved scan-time TF. This reuses the research localizer’s planar approximation; it assumes the requested object intersects the laser plane and ignores the small camera-to-laser translation when forming the ray.

The selected metric point is sent directly in `SemanticQueryResult`. The adapter computes a 0.5 m standoff along the current robot-to-target direction in the same frame. Nav2 uses the occupancy map, costmaps, and localization to plan and execute the approach. The coordinator tracks success/failure and automatically re-enables exploration after a delay. In the mapped chair demonstration, exploration nodes are intentionally disabled; the observed “exploration resumed” status is a coordinator state transition, not an exploration-motion test.

If the model returns an invalid or ambiguous box, or the sensor data and transforms cannot support localization, the command fails without sending a goal. The required message field `confidence=1.0` is a placeholder and should not be read as a model confidence score.

3 Red and blue chair demonstrations

A compact room contains two copies of the same course VisitorChair mesh with different red and blue materials. Both are visible from the starting pose and have the original solid collision geometry. For either command, the other chair serves as a distractor. The robot is reset between commands, and the map is available before grounding.



(a) Move to the red chair.

(b) Move to the blue chair.

Figure 1: Camera images from the two runs. LocateAnything selects the chair named in the command while the other chair remains visible.

Requested target	Inference (s)	To terminal (s)	Final distance (m)	Arrival ≤ 1.2 m
Red chair	0.168	8.079	1.080	Pass
Blue chair	0.184	7.405	1.170	Pass

Table 1: Final red and blue runs, with baseline detection and memory still running. Time to terminal includes grounding and navigation; inference time excludes the 4.697 s model load.

Both commands produced a box around the requested chair and led to that same chair during navigation. The selected map coordinates were (0.858, 0.727) for red and (0.776, -0.691) for blue. Source image-to-scan offsets were 94 ms and 10 ms. Selected-point discrepancies from the Gazebo chair origins were 0.339 m and 0.434 m. These include the difference between the observed near surface and the model origin; they are not direct estimates of sensor accuracy.

Arrival was checked using the robot and object world poses from Gazebo, sampled after navigation ended and before exploration resumed. The planar distance must be at most 1.2 m, as required by the course. The distance to the other chair was 1.623 m for the red run and 1.648 m for the blue run. Gazebo object poses are used only by this evaluator, never by grounding, target selection, or goal generation.

Four earlier attempts also passed, and all six runs are included in `results.json`. These runs demonstrate selection between the two colored chairs, but are too limited to establish a general success rate. The final blue run finished only 0.030 m inside the distance limit.

4 Results and discussion

All nine packages built successfully. The detector and integration checks passed, including 40 tests and a separate ROS navigation-cancellation test. The live detector self-test passed. Generic `go to person` reached the center person at 0.702 m. Commands for observed IDs `person_0` and `person_1` reached northwest and southeast people at 0.672 m and 0.688 m. The corrected trash-can run passed at 0.726 m. Exploration resumed after each successful baseline run.

The baseline chair test needed assistance. Unaided exploration for 600 s left the chair mislabeled as a trash can, and `go to chair` failed with no chair landmark. After an ordinary memory-based reposition command, an operator turned the camera toward the chair and held still for 22 s. The unchanged detector/memory then relabeled it; a new chair command reached the correct chair at 0.973 m and exploration resumed. The chair result therefore demonstrates navigation after assisted observation, not fully autonomous discovery. This used the normal memory update process, with no injected landmarks, ground-truth goals, or threshold changes.

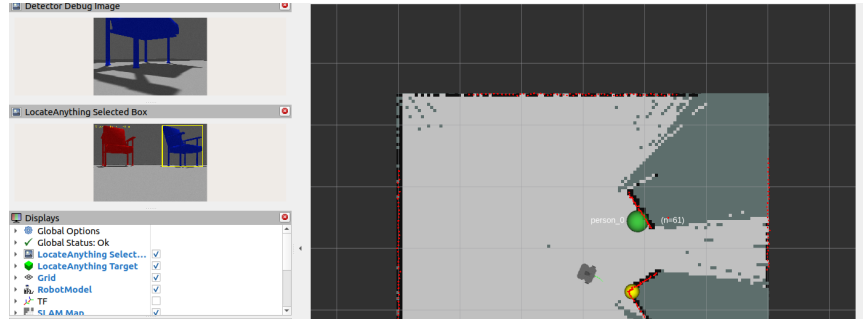


Figure 2: RViz after the second blue-chair run (cropped). The saved box and yellow marker show the selected chair. The green `person_0` marker is the baseline’s incorrect label for the red chair.

In the same colored-chair scene, both baseline commands failed with “no active chair in memory”. The detector labeled the red chair as a person. Although the baseline parser also ignores color, this comparison reflects detection and memory errors as well as the difference in language handling. The blue run additionally encountered an unavailable map transform in the evaluator.

Some earlier checks also failed. The first generic-person run had missing Gazebo pose queries; a later query of the stationary objects gave a supplemental distance of 0.751 m, with the original failure kept in the record. The first trash-can command selected a mislabeled chair and stopped 2.096 m from the intended trash can. A retry selected the right region but aborted (2.090 m). Waiting for cancellation acknowledgement and Nav2 idle fixed that handoff race; the later trash-can run passed. The failed attempts and subsequent checks are included in `results.json`.

My main contribution is connecting the full description and camera image to the target used by the existing navigation system. I completed the detector, added the persistent `LocateAnything` worker and synchronized localization, and created the colored-chair scene. I also corrected the robot-relative target selection and approach pose, and the cancellation and message-ordering issues encountered during testing. The project continues to use the course’s ROS interfaces and Nav2 planner.

Tests ran on Ubuntu 22.04 with ROS 2 Humble, an RTX 5090, PyTorch 2.7.1+cu128, Ultralytics 8.4.60, and NumPy 1.26.4. The current extension requires a mapped scene and a visible, stationary target that intersects the laser plane. Searching for unseen objects and reliable rejection of absent targets remain outside this project.

The [project repository](#) contains the code, report, and README launch instructions. Its `demo_video/` directory is the location for the recorded demonstration of *Move to the red chair*. and *Move to the blue chair*.